Intermediate DevOps
Day 2: Infrastructure as code and credentials

United Airlines · Pluralsight instructor-led training

Kangeyan Passoubady
9-11 November 2026

KavinSchool

DISCUSSION

Recall: Follow a change

Without opening your notes, write a short explanation:

1. How does a changed file reach the pipeline?
2. What prevents a failing change from merging to main?
3. Why can a passing pipeline still miss a defect?

Think privately for 60 seconds, then optionally share one sentence in chat. Compare with the next slide.

ANSWER

What you built on Day 1

- Harness Open Source running in Docker on your lab machine, with the `flightops` project and the `flight-status` repository.
- A CI pipeline in `.harness/ci.yaml` that lints the code and runs unit and API tests in containers.
- Integration tests that run against a Postgres background service.
- A branch rule on `main` that requires a pull request and a passing pipeline check.

Today you create the infrastructure the application will run on, and you manage the credentials it needs.

Day 2 agenda

MORNING SESSIONS — AM

1	RECAP	Day 1 recap
2	LECTURE	Infrastructure as code concepts
3	LECTURE	Terraform, CloudFormation and the AWS CDK
4	LAB 04	Terraform foundations on Kubernetes
5	LECTURE	Infrastructure as code in pipelines

AFTERNOON SESSIONS — PM

6	LAB 05	Terraform in the pipeline
7	LECTURE	Credential management
8	LAB 06	Managing credentials
9	WRAP-UP	Summary and questions

AWS: Amazon Web Services. CDK: Cloud Development Kit.

Infrastructure as Code Concepts

Declarative specifications, idempotency, and convergence

KavinSchool

How infrastructure was managed before code

The Snowflake Era

MANUAL & FRAGILE

- × Built and patched by hand from runbooks or memory
- × Multi-week ticketing queues for provisioning
- × Diverged into unique, unrepeatable "snowflake" servers
- × Slow disaster recovery: nobody knew full configuration

Programmable Cloud

DECLARATIVE & AUTOMATED

- ✓ Version-controlled declarations in Git repositories
- ✓ Pull request review gates before any environment change
- ✓ Automated idempotency and convergence across environments
- ✓ Disposable & reproducible: rebuilt in minutes via APIs

THE TURNING POINT

AWS EC2 (2006) transformed infrastructure from physical procurement into programmable API calls — turning servers into disposable, versioned, and auditable software artifacts.

What infrastructure as code means

Infrastructure as code applies software development practices to infrastructure (Morris, 2020).

- Definitions of servers, networks, clusters and permissions are files in version control.
- Every change is a commit with an author, a date and a reason.
- Changes are reviewed in pull requests before anyone applies them.
- Automation applies each change the same way, in every environment.

The files describe the infrastructure you intend to have. Lab 04 shows what happens when the real infrastructure differs.

Kinds of infrastructure as code tools

Category	What it manages	Examples
Provisioning	Cloud and platform resources, through their APIs	Terraform, OpenTofu, AWS CloudFormation, AWS CDK, Pulumi
Configuration management	Packages, files and services on existing servers	Ansible, Chef, Puppet, CFEngine
Image building	Machine images and container images	Packer, Dockerfiles
Kubernetes objects	Workloads and settings inside a cluster	YAML manifests, Helm, Kustomize

YAML: YAML Ain't Markup Language.

This course uses Terraform for provisioning today, and Kubernetes manifests on Day 3.

Imperative and declarative instructions

Imperative

You list the commands, in order. You decide what to do if a step already happened.

```
kubectl create namespace flightops-dev
kubectl create quota dev-quota \
  --hard=pods=10 -n flightops-dev
```

Declarative

You describe the result. The tool compares it with what exists and works out the steps.

```
resource "kubernetes_namespace_v1" "dev" {
  metadata {
    name = "flightops-dev"
  }
}
```

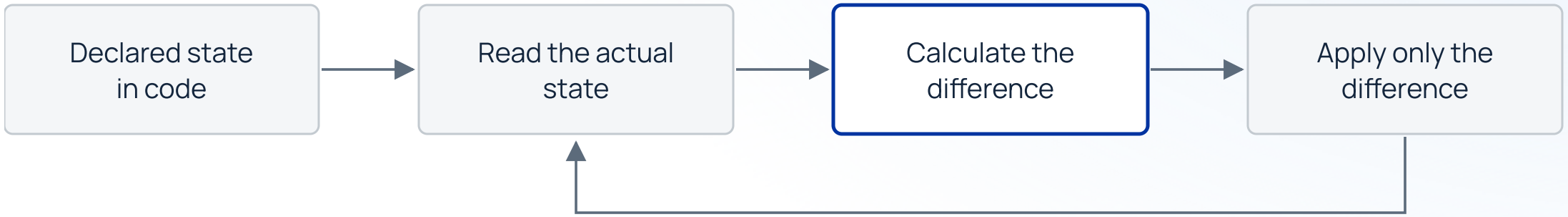
Idempotency

An operation is idempotent when running it many times has the same effect as running it once.

Command	First run	Second run
`kubectl create namespace dev`	Creates the namespace	Fails with AlreadyExists
`kubectl apply -f namespace.yaml`	Creates the namespace	Reports "unchanged"
`terraform apply`	Creates the namespace	Reports "No changes"
`mkdir -p /opt/app`	Creates the folder	Does nothing

You can re-run an idempotent operation after a failure or a timeout without side effects.

Desired state and convergence



Repeat on every run. If nothing differs, nothing changes.

Mark Burgess designed CFEngine in 1993 around this idea. Each run moves a system closer to the declared state. Terraform plans and Kubernetes controllers work the same way.

Press ↓ for supporting detail: Declarative vs. Imperative IaC

REFERENCE ONLY

DEEP DIVE / WHAT IS IAC?

Declarative vs. Imperative Infrastructure

IMPERATIVE (SCRIPTING "HOW")

Step-by-Step Procedure

Explicit commands specify each operation: "create subnet, wait 10s, provision VM". If a script fails midway, rerunning it causes duplicate name collisions unless you write complex defensive rollback logic.

DECLARATIVE (DECLARING "WHAT")

Target End State

You describe the desired result in code: "a VPC with three subnets and two VMs". The tool (Terraform, CloudFormation) computes the current state, calculates the delta, and executes only what is missing or changed.

WHY CONSOLE CLICKS FAIL IN TEAMS

Manual console clicks create undocumented, untracked "snowflake" environments. Declaring infrastructure in version-controlled code ensures peer review, automated testing, disaster repeatability, and drift detection.

Configuration drift

`Git Code (Intended) ≠ Cloud State (Actual)`

Drift: Any divergence between declared code and live infrastructure. Detected via `terraform plan` refresh or CloudFormation drift detection.

- **Console ClickOps:** Manual edits directly in cloud/K8s web consoles
- **2 AM Hotfixes:** Urgent production patches never backported to Git
- **Rogue Automation:** Ad-hoc scripts modifying live resources outside IaC

Path A: Revert to Code

OVERWRITE

Run `terraform apply` to overwrite and destroy unauthorized manual changes, restoring reality to the declared Git baseline.

Path B: Codify into Git

ADOPT

Submit a pull request adopting the manual change into Terraform code, keeping intent, review history, and cloud reality synchronized.

Drift Prevention Rule: Revoke human console write access in production environments — only authenticated CI/CD pipelines hold credentials to apply changes.

Mutable and immutable infrastructure

Mutable

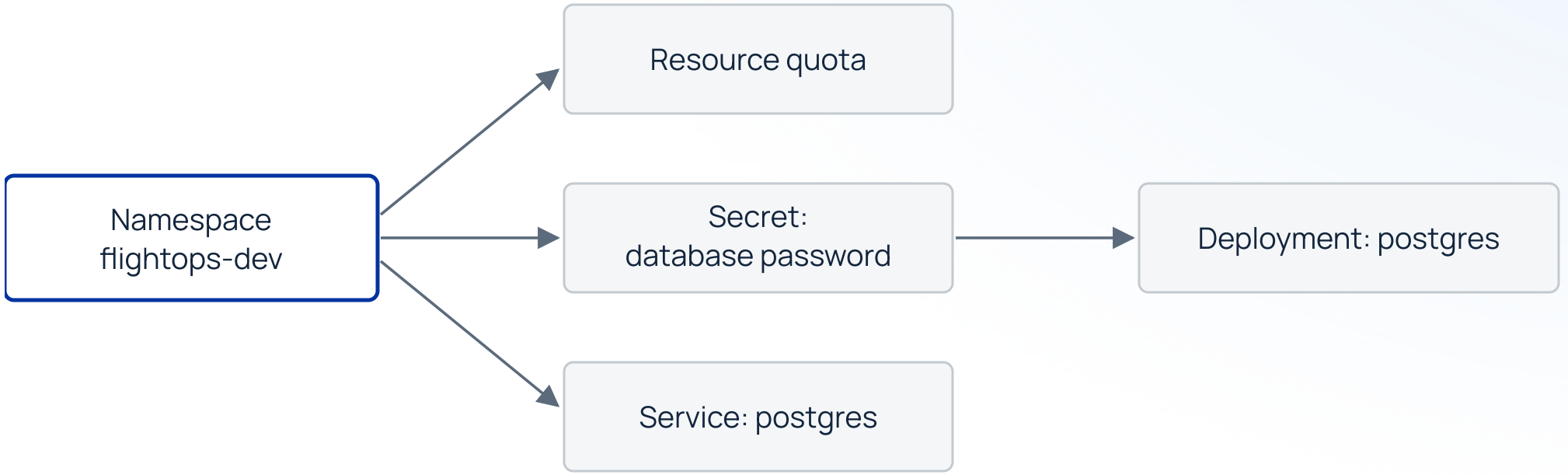
- Each server is updated in place.
- Its state depends on every change made since it was built.
- Rollback means undoing changes one by one.

Immutable

- A change produces a new image. New servers or containers replace the old ones.
- Nothing is modified after deployment.
- Rollback means deploying the previous image again.

Containers and Kubernetes Deployments follow the immutable model. You will use it on Day 3.

The dependency graph

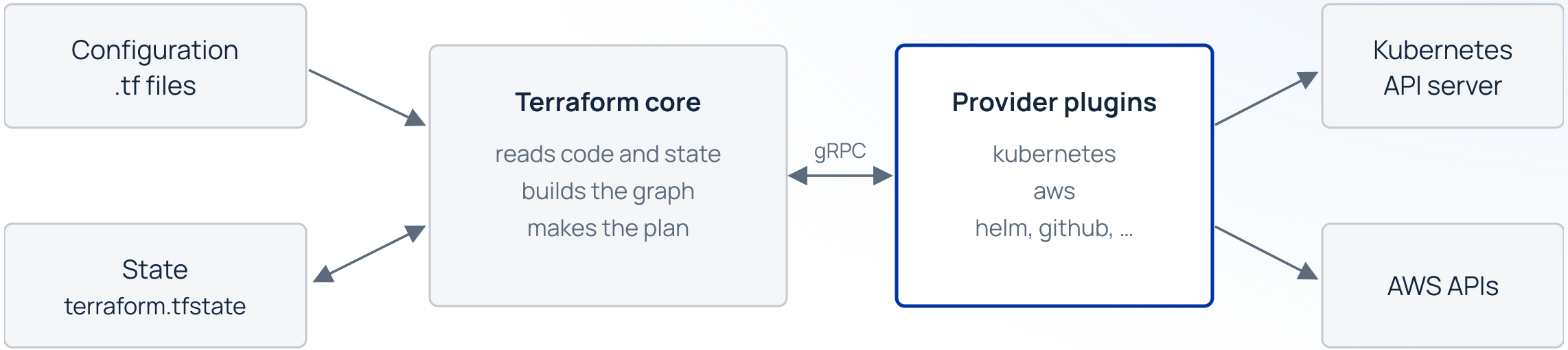


An arrow means the first object must exist before the second. Terraform builds this graph from references in the code. It creates independent objects in parallel, up to 10 at a time by default, and destroys in reverse order.

Terraform, CloudFormation and CDK

State management, resource graphs, and cloud tooling

How Terraform is built



Terraform core has no knowledge of Kubernetes or AWS. Providers are separate programs that turn resources into API calls. `terraform init` downloads them from the Terraform Registry.

Press ↓ for supporting detail: What is Terraform?

REFERENCE ONLY

DEEP DIVE / CORE ARCHITECTURE

What is Terraform? Core vs. Providers

TERRAFORM CORE

The Orchestration Engine

A single compiled binary. It parses HashiCorp Configuration Language (HCL), constructs the dependency graph, inspects previous state, and calculates what resources need creation, update, or destruction.

PROVIDER PLUGINS

Platform Translators

Standalone executables (AWS, Kubernetes, Azure) maintained independently. Core communicates with them over gRPC, asking providers to execute the vendor API requests required to reach the target state.

CLEAN SEPARATION OF CONCERNS

Terraform Core has zero cloud knowledge built in. When AWS releases a new service or Kubernetes adds an API, only the respective provider plugin needs updating—your Terraform CLI binary stays unchanged.

The Terraform language

HCL: HashiCorp Configuration Language.

```
variable "environments" {
  type    = set(string)
  default = ["dev", "prod"]
}
resource "kubernetes_namespace_v1" "env" {
  for_each = var.environments
  metadata {
    name = "flightops-${each.key}"
  }
}
output "namespaces" { value = keys(kubernetes_namespace_v1.env) }
```

- `variable` is an input value.
- `resource` is an object Terraform manages. Its address is the type and the name.
- `for_each` makes one instance per item, such as `kubernetes_namespace_v1.env["dev"]`.
- `output` prints a value after apply and lets other code read it.

Provider versions and the lock file

```
terraform {
  required_version = ">= 1.17"
  required_providers {
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = "~> 3.4"
    }
  }
}
```

- `source` names the provider in the Terraform Registry.
- `~> 3.4` accepts version 3.4 or any later 3.x release. It rejects 4.0.
- `terraform init` records the selected version and its checksums in `.terraform.lock.hcl`. Commit that file.

Why Terraform keeps state

- State maps each resource address to a real object (e.g. `kubernetes_namespace_v1.env["dev"]` to `flightops-dev`).
- State records dependencies, so Terraform deletes objects in the right order after code removal.
- State stores last known attribute values, which plans compare against.
- By default, state is a JSON file named `terraform.tfstate`.

If state is lost, Terraform no longer knows which objects it created.

Press ↓ for supporting detail: The State File Mental Model

REFERENCE ONLY

DEEP DIVE / MENTAL MODEL

The State File Mental Model

1. IDENTITY MAPPING

Code to Cloud IDs

Cloud APIs only know generated IDs (e.g. `i-0abc1234def`). State maps human-readable code addresses (`aws_instance.web`) to actual physical cloud entities.

2. DRIFT DETECTION

Spotting Rogue Edits

State stores last-known attributes. During `plan`, Terraform refreshes against live APIs to pinpoint out-of-band manual changes made in web consoles.

3. SAFE DELETIONS

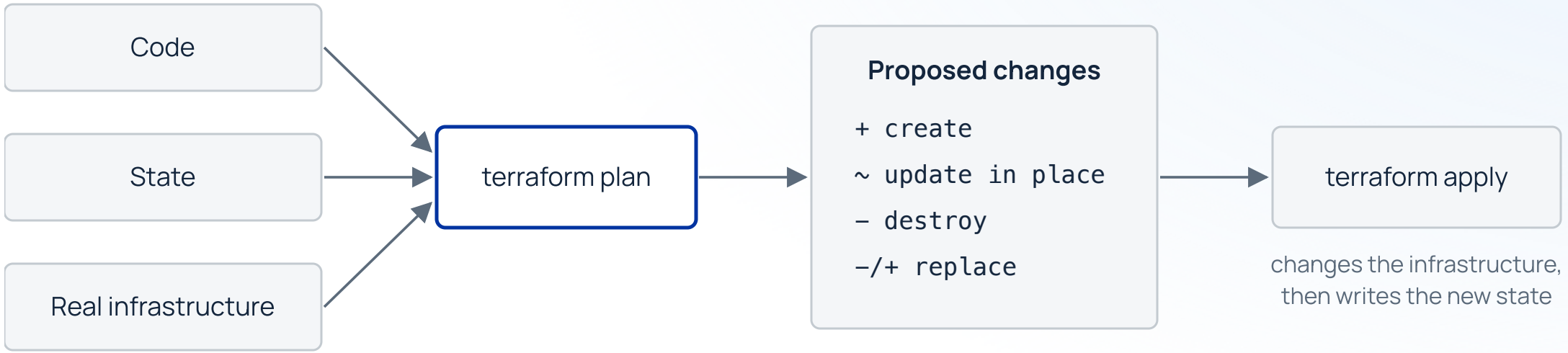
Dependency Memory

When you delete code, the code is gone! State remembers that the object existed and what depended on it, allowing clean, reverse-order destruction.

WHY CAN'T TERRAFORM JUST QUERY THE CLOUD?

Cloud APIs cannot tell which team or Terraform workspace created an arbitrary security group or VPC, nor can they track resources that were just deleted from code without a state record.

Plan and apply



Nothing changes until apply runs. `terraform plan -out=tfplan` saves a plan, and `terraform apply tfplan` applies that saved plan without recalculating it.

Modules

```
module "postgres" {
  source   = "./modules/postgres"
  for_each = var.environments

  namespace = kubernetes_namespace_v1.env[each.key].metadata[0].name
}

# modules/postgres/variables.tf
variable "namespace" {
  type = string
}
```

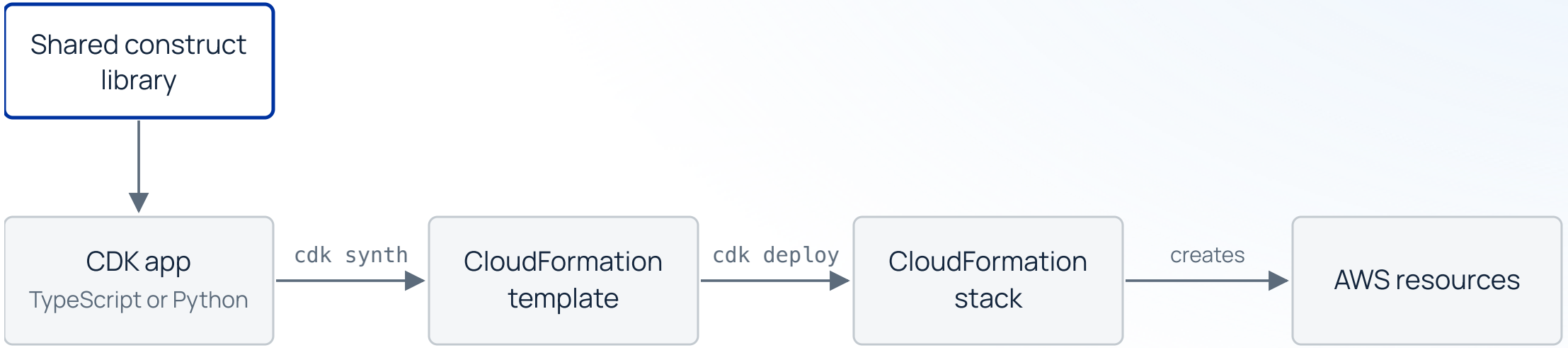
- A module is a folder of `.tf` files with input variables and outputs.
- The root module calls child modules and passes values to them.
- With `for_each`, one module builds the same database in dev and in prod.
- Modules can come from a local folder, a Git repository or a registry.

AWS CloudFormation

- You describe AWS resources in a JSON or YAML template.
- CloudFormation creates them as a stack and keeps the state inside AWS.
- A change set previews an update before you run it.
- If an update fails, CloudFormation rolls the stack back.
- Drift detection compares a stack with the real resources.

```
Resources:
  FlightDataBucket:
    Type: AWS::S3::Bucket
    Properties:
      VersioningConfiguration:
        Status: Enabled
```

The AWS Cloud Development Kit



CDK code produces a CloudFormation template, and CloudFormation creates the resources. Constructs are reusable classes that bundle resources with defaults. At United, teams use a shared CDK library. A library like this packages approved resource settings as constructs.

Terraform, CloudFormation and the CDK compared

	Terraform	CloudFormation	AWS CDK
Language	HCL	JSON or YAML	TypeScript, Python, Java, C#, Go
State	A state file in a backend you choose	Kept by AWS in the stack	Kept by AWS, through CloudFormation
Preview	`terraform plan`	Change set	`cdk diff`
Targets	Any API with a provider, such as AWS, Kubernetes or GitHub	AWS	AWS
Reuse	Modules	Nested stacks and modules	Constructs

Declared state, previews, review in pull requests and automated apply work the same way in all three.

HANDS-ON LAB

Lab 04: Terraform Foundations on Kubernetes

Day 2 · 45–60 minutes

HANDS-ON LAB

Lab 04: Terraform foundations on Kubernetes

Open `labs/04-terraform-foundations.md` in the course repository.

- Install k3d, kubectl and Terraform, and create the local Kubernetes cluster.
- Write Terraform for dev and prod namespaces with quotas and default limits.
- Run plan and apply, make a manual change, and find the drift.
- Add a Postgres database per environment with a module, and push the code to Harness.

Infrastructure as Code in Pipelines

Automated plan and apply workflows, state locking, and drift

Why apply infrastructure code from a pipeline

When engineers apply from their laptops:

- Each laptop can have a different Terraform or provider version.
- State files are kept on laptops, or two people apply at the same time.
- Cloud credentials are copied to many machines.
- Nobody reviews the change, and there is no record of who applied it.

A pipeline runs one pinned version, uses one remote state, holds the credentials in one place, and applies only after review.

Remote state and locking

Race Condition Risk

CONCURRENCY CLASH

Without distributed locking, two team members or pipeline jobs running `apply` simultaneously create catastrophic race conditions:

- × Both runs read the identical baseline state at execution start
- × Second write overwrites and clobbers first run's state updates
- × State mapping corrupts; resources orphan or duplicate in cloud

Distributed Lock Mesh

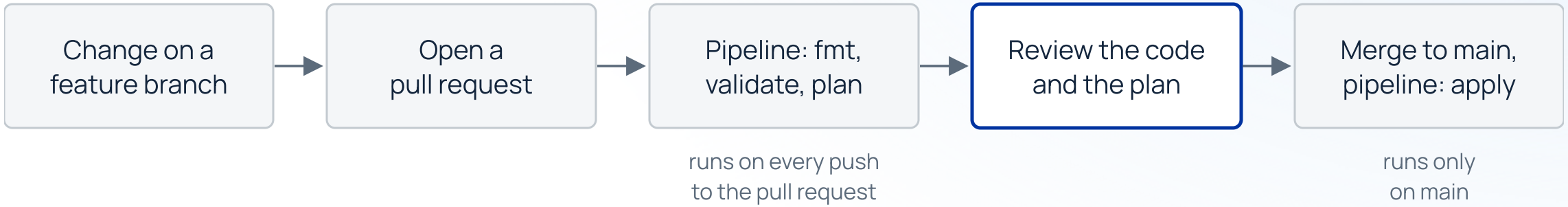
REMOTE SHIELD

Remote backends enforce mutual exclusion through coordinated locking before any modification begins:

1. **Remote Storage:** State saved centrally in AWS S3, HCP Terraform, or K8s Secret (Lab 05).
2. **Lock Acquisition:** Acquires lock ID in DynamoDB or K8s Lease; Run B fails with "Error acquiring the state lock".
3. **Atomic Release:** Lock is released only after state successfully flushes to backend.

Golden Rule: Never edit state files by hand. Remote state is Terraform's single authoritative source of truth — use `terraform force-unlock` only after confirming no process is running.

Plan on pull request, apply on merge



The branch rule from Lab 03 also protects `main` in the infrastructure repository. Reviewers approve the code change and its plan output together.

Infrastructure through code (General DevOps)

REAL-WORLD SCENARIO

GitLab: managing GitLab.com infrastructure.

- GitLab's engineering handbook describes Terraform configuration for production, staging and operations environments.
- Shared configuration keeps staging and production topology aligned; variables hold environment-specific sizing.
- Course connection: review infrastructure changes in version control and apply them through CI.

Source: GitLab handbook, infrastructure management.

Infrastructure through code (Retail)

SKIP | REAL-WORLD SCENARIO

Wayfair: self-service cloud environments with Terraform.

- Wayfair described a Terraform module that creates development and production cloud projects with foundational settings.
- Its sandbox allowed manual prototyping, while development and production changes had to go through infrastructure as code.
- Course connection: experimentation and controlled environments can have different change paths.

Source: Wayfair's HashiCorp presentation.

Reviewing a plan

- Read the summary first, for example "Plan: 1 to add, 2 to change, 0 to destroy."
- Look for destroy (`-`) and replace (`-/+`) actions. Replacing a database resource can delete its data.
- Check that the plan changes only the resources the pull request meant to change.
- A saved plan is valid only for the state it was made from. If the state changes first, Terraform rejects the plan as stale.

State is sensitive data

- State stores every attribute of every resource in plain text, including generated passwords and tokens.
- `sensitive = true` hides a value in plan output. The value is still written to state.
- Anyone who can read the state can read those values.
- Protect the backend with encryption at rest, and give access only to the pipeline and a few administrators.

In Lab 05 the state is stored in a Kubernetes Secret. In Lab 06 you find a database password in it.

Policy as code

A policy is a rule written as code and checked automatically. This Open Policy Agent (OPA) rule, written in Rego, rejects a plan that deletes a namespace.

```
package terraform.namespaces

deny contains msg if {
  some rc in input.resource_changes
  rc.type == "kubernetes_namespace_v1"
  "delete" in rc.change.actions
  msg := sprintf("%s would be deleted", [rc.address])
}
```

The input is the plan as JSON, from `terraform show -json tfplan`. Other policy engines include HashiCorp Sentinel and AWS CloudFormation Guard. The Harness platform uses OPA policies to check pipelines and other resources.

GitOps principles

Principle	Meaning
Declarative	The desired state of the system is described declaratively.
Versioned and immutable	The desired state is stored with a complete version history.
Pulled automatically	Software agents pull the desired state from the source.
Continuously reconciled	Agents compare the actual state with the desired state and correct differences.

Lab 05 uses a push model, where the pipeline applies changes. Pull-based tools such as Argo CD and Flux run inside the cluster. Harness offers a GitOps module based on Argo CD.

Detecting drift from a pipeline

- Run a plan on a schedule, for example every night.
- With `-detailed-exitcode`, plan exits with 0 for no changes, 2 for changes and 1 for an error.
- On exit code 2, alert the team. Decide whether to revert the change or update the code.

```
rc=0
terraform plan -detailed-exitcode -input=false || rc=$?
if [ "$rc" -eq 2 ]; then echo "Drift found"; exit 1; fi
exit "$rc"
```

The `|| rc=$?` keeps the step's `set -e` shell from stopping before the check.

KNOWLEDGE CHECK

Section Quiz 1: Terraform in Pipelines

Twelve questions. Five minutes, including feedback.

- Use the Kahoot game PIN shared by your instructor.
- After each answer, compare the reasoning with your own.

HANDS-ON LAB

Lab 05: Terraform in the Pipeline

Day 2 · 35–45 minutes

HANDS-ON LAB

Lab 05: Terraform in the pipeline

Open `labs/05-terraform-pipeline.md` in the course repository.

- Move the Terraform state to the Kubernetes backend.
- Store a cluster credential for the pipeline as a Harness secret.
- Build a pipeline that runs fmt, validate and plan on pull requests, and apply on `main`.
- Change an environment through a pull request and watch the apply.

Credential Management

Secrets hygiene, injection boundaries, and role-based access

What counts as a credential

A credential is any value that proves an identity or grants access.

- Passwords and database connection strings
- API tokens, such as your Harness personal access token
- Secure Shell (SSH) and Transport Layer Security (TLS) private keys
- Cloud access keys, such as an AWS access key ID and secret access key
- kubeconfig files and Kubernetes ServiceAccount tokens

For each credential, know what it grants access to, who holds a copy, and when it expires.

Where secrets leak

Git History
SOURCE CODE

Passwords committed and later deleted remain permanently accessible in commit DAG history.

Pipeline Logs
CI/CD OUTPUT

`echo $TOKEN` or unmasked `set -x` prints credentials to shared build stdout logs.

Image Layers
CONTAINERS

Secrets copied via `COPY` in layer 1 persist in image blobs even if deleted by layer 2.

State Files
TERRAFORM

Terraform `.tfstate` files store generated passwords and connection strings in plain text.

Process Data
SHELL & /PROC

`docker inspect` and `/proc` expose runtime environment variables to local host users.

Chat & Tickets
COLLABORATION

Credentials casually pasted into Slack, Teams, or Jira tickets to quickly unblock a colleague.

Defense in Depth: Deleting a secret does not remove it from history or backups. Any exposed credential must be revoked and rotated immediately.

The secret-zero problem

The Static Bootstrap Dilemma

THE SECRET-ZERO TRAP

To retrieve dynamic credentials from HashiCorp Vault or AWS Secrets Manager, a pipeline must first authenticate to the store. If this bootstrap key is a static token or long-lived API secret, it **is** Secret Zero — a single point of failure that can leak across logs, images, and history.

Tokenless Cryptographic Attestation

WORKLOAD IDENTITY

Instead of storing bootstrap passwords, the execution platform cryptographically proves who the running workload is:

OIDC Federated Trust
CI pipelines mint ephemeral OpenID Connect JWTs exchanged directly with cloud IAM for temporary tokens.

Cloud IAM Roles (IRSA)
EKS Pods and Harness delegates assume IAM roles directly via AWS STS without any stored access keys.

K8s ServiceAccounts
API server cryptographically signs time-bound, audience-scoped JWT tokens projected into Pod volumes.

Core Principle: Never store static credentials to unlock other credentials. Eliminate Secret Zero entirely by moving to platform-attested workload identities.

Least privilege

"Every program and every user of the system should operate using the least set of privileges necessary to complete the job" (Saltzer and Schroeder, 1975).

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deployer
  namespace: flightops-dev
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "patch", "update"]
```

- Allow only the verbs the job needs.
- Limit the namespace and the resource types.
- Give each pipeline its own identity.
- Prefer credentials that expire.

Rotation and revocation



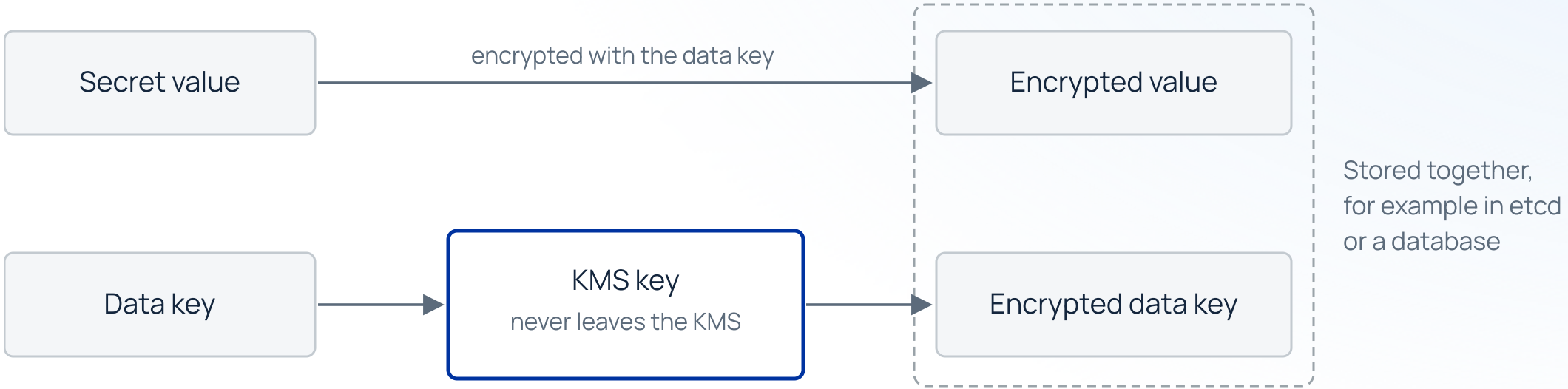
Rotation replaces a credential with a new value. Revocation makes the old value stop working. Rotate on a schedule, and at once after a suspected leak. Update every consumer before you revoke the old value.

Short-lived and dynamic credentials

Mechanism	Lifetime	Example
Kubernetes bound ServiceAccount token	Chosen per request, for example one hour	`kubectl create token deployer --duration=1h`
AWS Security Token Service (STS) temporary credentials	15 minutes to 12 hours	An IAM role assumed by a pipeline or a pod
Vault dynamic database credentials	A lease, for example one hour	A new database user per request, removed when the lease ends

A leaked short-lived credential stops working on its own. A long-lived credential works until someone revokes it.

Encryption at rest and envelope encryption



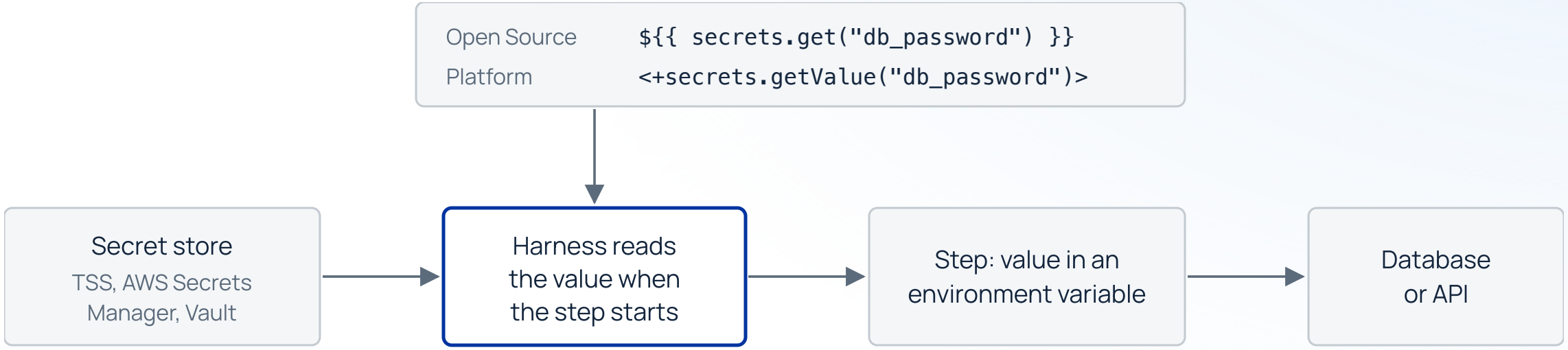
A data key encrypts the value. A key in a key management service (KMS), such as AWS KMS, encrypts the data key. The KMS key never leaves the KMS, and rotating it does not require re-encrypting the stored data.

Kubernetes Secrets

```
$ kubectl get secret db -n flightops-dev -o jsonpath='{.data.password}'
UzNjcjN0LVZhbHVl
$ echo UzNjcjN0LVZhbHVl | base64 -d
S3cr3t-Value
```

- The API server stores Secrets in etcd, base64-encoded. Anyone can decode base64.
- Encryption at rest is a separate API server setting. Amazon EKS can encrypt Secrets with a key in AWS KMS.
- Anyone allowed to read Secrets in a namespace, or to create Pods there, can read the values.

Secret stores and pipelines



The pipeline YAML holds a reference to the secret. At United, the enterprise store is TSS. The Harness platform reads external stores through secret manager connectors. Harness Open Source keeps secrets in its own database.

Press ↓ for supporting detail: Enterprise Secrets Managers

REFERENCE ONLY

DEEP DIVE / CREDENTIAL SECURITY

What is an Enterprise Secrets Manager?

THE VULNERABILITY

Hardcoded & Static Secrets

Hardcoding passwords or storing plain text in files and environment variables leaves permanent audit leaks in Git commits, container layers, and shell history where they sit unrotated for months.

THE SOLUTION

Runtime Dynamic Injection

Enterprise vaults (TSS, HashiCorp Vault, AWS Secrets Manager) securely encrypt secrets, enforce access control, log every retrieval, and vend short-lived credentials directly to running jobs.

SECRETS INJECTION BOUNDARY

Pipeline definitions store only references (e.g. `<+secrets.getValue("db_password")>`). The Harness Delegate retrieves the secret in memory when the step begins and purges it immediately on completion.

Delivering secrets (General DevOps)

REAL-WORLD SCENARIO

GitLab: its documented HashiCorp Vault integration.

- A GitLab CI job can receive a signed identity token and use it to authenticate to Vault.
- Vault checks the token and its role constraints before granting access to the requested secret.
- Course connection: keep secret values out of pipeline definitions and limit which workloads can retrieve them.

Source: GitLab, authenticating and reading secrets with Vault.

Delivering secrets (Retail)

SKIP | REAL-WORLD SCENARIO

Shopify: encrypted application secrets, published in 2015.

- Shopify introduced EJSON to deliver API keys and database passwords to containerized applications.
- Public-key encryption let developers add or rotate encrypted values without access to the decryption key.
- Course connection: protect the decryption key separately. Encoding a Kubernetes Secret is not encryption.

Source: Shopify, Secrets at Shopify: Introducing EJSON.

Secret scanning and log masking

Secret scanning

- Looks for secret patterns, such as AWS access keys, in commits.
- A pre-commit hook, such as gitleaks, blocks the commit on a laptop.
- Harness Open Source can reject a push and name the file and line.
- Custom secret formats can pass, because scanning relies on patterns.

Log masking

- Replaces known secret values with asterisks in logs.
- The Harness platform masks secrets. Harness Open Source 3.3.0 does not.
- A changed value, such as base64 or part of a string, is not masked.

Do not print secrets in logs. Scanning and masking only catch some mistakes.

KNOWLEDGE CHECK

Section Quiz 2: Credentials

Twelve questions. Five minutes, including feedback.

- Use the Kahoot game PIN shared by your instructor.
- After each answer, compare the reasoning with your own.

HANDS-ON LAB

Lab 06: Managing Credentials

Day 2 · 30–40 minutes

HANDS-ON LAB

Lab 06: Managing credentials

Open `labs/06-credentials.md` in the course repository.

- Find credentials on disk and in Terraform state, and decode a Kubernetes Secret.
- Store a secret in Harness and see that Harness Open Source does not mask it in logs.
- Create a least-privilege deploy ServiceAccount and test its permissions.
- Block a leaked key with secret scanning, then rotate a token.

DISCUSSION

Credential boundaries: Discussion

Explain the dev deployer credential to a new teammate without showing any secret values.

1. Think privately for 30 seconds: what can it change, and what evidence checks that boundary?
2. In pairs, explain for 60 seconds. The listener asks: what proves an old credential no longer works?
3. Revise one sentence naming the identity, its allowed scope and the verification.

Solo written explanations are equally valid. Do not share tokens, kubeconfigs or production details.

RECAP

Day 2 summary

MODULE 1

Declarative IaC

- Desired state defined in version-controlled Git files
- Idempotent execution engines (Terraform, CloudFormation)
- Automated graph resolution of cloud dependencies
- Immutable infrastructure over in-place snowflake mutation

MODULE 2

Terraform Workflow

- Three-way comparison: Code vs. State vs. Real Cloud
- Plan generated on PR; apply executed on main merge
- Encrypted remote backend storage (S3 / HCP / K8s)
- Distributed state locking prevents concurrency collisions

MODULE 3

Delivery Engine

- Dedicated CI runners with pinned tool versions
- Scheduled drift detection catches out-of-band ClickOps
- Drift remediation choice: Revert change vs. Codify into Git
- Shift-left policy as code validation gates (OPA / Conftest)

MODULE 4

Secret Hygiene

- Least privilege permissions with minimal blast radius
- Central secret managers; never leak values in build logs
- Eliminate Secret Zero via OIDC and Workload Identity
- Mandatory rotation: verify old token revoked & new active

TOMORROW · DAY 3 **Containers, Kubernetes Orchestration & Continuous Delivery Pipelines**

References

- Burgess, M. (1995). Cfengine: a site configuration engine. USENIX Computing Systems 8(3).
- Fowler, M. (2012). SnowflakeServer and PhoenixServer. martinfowler.com/bliki
- Fowler, C. (2013). Trash your servers and burn your code: immutable infrastructure and disposable components. chadfowler.com
- Morris, K. (2020). Infrastructure as Code, 2nd ed. O'Reilly.
- HashiCorp. Terraform documentation: state, backends, plan, modules. developer.hashicorp.com/terraform
- AWS. CloudFormation User Guide and AWS CDK Developer Guide. docs.aws.amazon.com
- OpenGitOps (2021). GitOps Principles v1.0.0. opengitops.dev
- Open Policy Agent documentation. openpolicyagent.org
- OWASP (Open Worldwide Application Security Project). Secrets Management Cheat Sheet. cheatsheetseries.owasp.org
- Saltzer, J. H., and Schroeder, M. D. (1975). The protection of information in computer systems. Proceedings of the IEEE 63(9).
- Kubernetes documentation: Secrets, encrypting data at rest, service account tokens, RBAC (role-based access control). kubernetes.io/docs
- AWS. IAM User Guide: temporary security credentials. docs.aws.amazon.com
- HashiCorp. Vault documentation: database secrets engine. developer.hashicorp.com/vault
- Harness documentation: secrets, secret managers, policy as code, open source pipelines. developer.harness.io

Let’s Connect

Scan the QR code or click the link below to stay in touch



Kangeyan Passoubady

linkedin.com/in/kpassoubady

Point your phone camera at the QR code

Thank You

Intermediate DevOps · Day 2: Infrastructure as Code and Credentials

KavinSchool